

Bank of New York — Production Readiness, Blue-Green Cutover & Incident Recovery Sprint

Continuation of the previous Bank of New York case study: *Real-Time Merchant Settlement Intelligence Platform*

1. Sprint Objective

The previous sprint produced:

```
Source Files
  ↓
Bronze
  ↓
Silver
  ↓
Gold
  ↓
FastAPI
  ↓
HTML/JavaScript Dashboard
```

The data engineers now needs to prove that this solution is **production-ready**.

The day's challenge has three phases:

Morning

Make the existing Bank of New York solution production-ready, ensure the complete test suite is green, pass security gates, and deploy it through GitLab CI with Jenkins as the secondary build path.

Midday

Now deliberately breaks something in production.

The data engineers must:

```
Detect
  ↓
Investigate
  ↓
```

```
Assess impact
↓
Decide
↓
Rollback
↓
Verify
↓
Produce evidence
```

Afternoon

Present the production solution and incident response to leadership, followed by a structured 360° peer review.

2. Starting Point

The data engineers starts the day with the **Bank of New York Settlement Intelligence Platform** from the previous case.

The existing repository contains:

```
straive-settlement-platform/
├── docs/
├── data/
├── sql/
├── pipeline/
├── api/
├── frontend/
├── tests/
├── Dockerfile
├── requirements.txt
└── .gitlab-ci.yml
```

The solution already supports:

- payment transaction processing
- settlement processing
- merchant risk
- Bronze/Silver/Gold
- settlement KPIs
- FastAPI
- Pydantic
- HTML/JavaScript dashboard
- automated tests

The data engineers's responsibility today is **not to rebuild the application**.

They must make it deployable, observable, recoverable and demonstrably production-ready.

3. Production Requirements

Bank of New York defines the following production requirements.

Requirement	Target
Test suite	100% required tests passing
Critical security findings	0
API health	HTTP 200
API response	<500 ms under normal load
Deployment	GitLab CI
Secondary build path	Jenkins
Deployment strategy	Blue-Green
Rollback	Required
Production secrets	No hard-coded secrets
Production smoke test	Mandatory
Data reconciliation	Mandatory
Evidence	Mandatory

4. Morning Sprint — Production Deployment

Task 1 — Production Readiness & Test Gate

The data engineers must first prove that the existing application is ready to deploy.

Run:

Unit Tests
↓
Data Model Tests
↓
Pipeline Tests
↓
API Tests
↓
Business Rule Tests
↓
Security Tests

The candidate must demonstrate:

Functional tests

- settlement calculation
- settlement gap
- SLA calculation
- merchant exception logic
- late-arriving event handling

Data tests

- fact-table grain
- duplicate detection
- referential integrity
- transaction/settlement reconciliation
- invalid data handling

API tests

200 valid request
400 invalid request
404 invalid merchant
422 invalid parameter

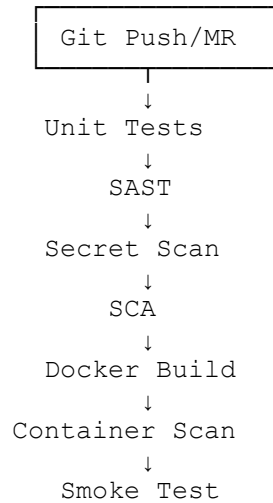
Mandatory negative test

The data engineers must prove that the system does **not** calculate an incorrect settlement rate when multiple settlement records exist for one transaction.

5. Task 2 — Security Gate

The data engineers must integrate or execute the security checks required by the previous module.

Pipeline:



Required checks

SAST

Identify:

- insecure SQL
- unsafe input handling
- insecure code patterns

Secret scanning

The repository must contain:

NO API keys
NO passwords
NO cloud credentials
NO database passwords

SCA

Check application dependencies.

The data engineers must identify:

- vulnerable dependency
- severity
- whether it blocks production
- remediation approach

Container scan

Scan the Docker image.

6. Task 3 — GitLab CI Production Pipeline

The data engineers must complete the GitLab pipeline.

Required flow:

```
validate
↓
test
↓
security
↓
build
↓
package
↓
deploy_dev
↓
smoke_test
↓
deploy_prod
```

Production deployment must require a controlled approval.

Example:

```
deploy_prod:
  stage: deploy
  when: manual
```

The data engineers must demonstrate appropriate use of:

- stages
 - jobs
 - artifacts
 - cache
 - rules
 - needs
 - protected variables
 - environments
 - manual approval
-

7. Task 4 — Jenkins Secondary Build Path

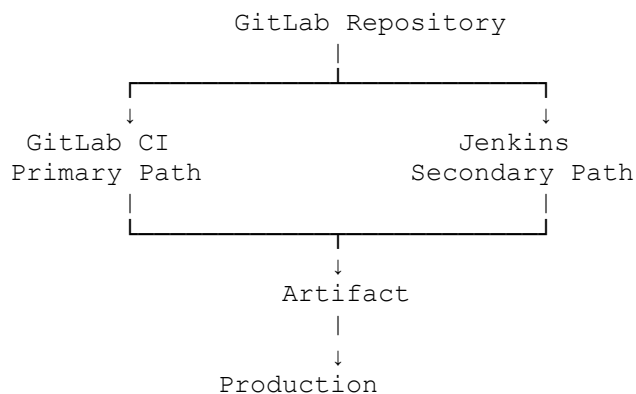
Bank of New York's engineering organisation maintains Jenkins for legacy enterprise workloads.

The data engineers must create a **Jenkinsfile** that can independently:

```
Checkout
↓
Install Dependencies
↓
Run Tests
↓
Security Checks
↓
Build Docker Image
↓
Publish Artifact
```

Jenkins is **not the primary production deployment path**.

The architecture is:



The data engineers must explain:

Why would an enterprise maintain Jenkins alongside GitLab CI?

8. Task 5 — Blue-Green Production Deployment

Bank of New York currently has:

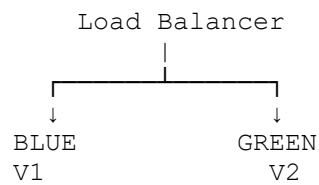
BLUE

Version 1

The data engineers deploys:

GREEN
Version 2

Architecture:



Initially:

100% traffic → BLUE

After successful validation:

100% traffic → GREEN

The data engineers must perform:

Pre-cutover checks

- application health
- API health
- database connectivity
- pipeline health
- test suite
- smoke test
- KPI reconciliation

Smoke test

```
GET /health
GET /api/v1/settlement-summary
GET /api/v1/merchant-exceptions
```

Verify:

```
HTTP 200
Valid JSON
Correct KPI values
Response time acceptable
```

Only after successful validation:

BLUE → GREEN

9. Production Evidence Package

Before moving to the incident phase, the data engineers must capture:

```
01_test_results
02_security_results
03_gitlab_pipeline
04_jenkins_pipeline
05_docker_image
06_deployment_version
07_blue_green_status
08_smoke_test_results
09_kpi_reconciliation
```

The important point is:

"It worked" is not sufficient. The data engineers must prove it worked.

10. MIDDAY — Injected Production Incident

Now deliberately breaks production.

Incident scenario

After the blue-green cutover, the operations dashboard starts showing:

Settlement Rate: 103.7%

Previously:

Settlement Rate: 95.2%

At the same time, operations reports that:

"Several merchants are showing settlement amounts greater than their successful transaction amounts."

The API is still returning:

HTTP 200

Therefore, a simple health check does **not** identify the problem.

11. Incident Task — Detect, Decide, Recover

The data engineers must treat this as a real production incident.

Step 1 — Detect

Identify the abnormality.

They should compare:

Previous production
vs
Current production

Investigate:

- API response
 - application logs
 - deployment version
 - KPI results
 - database/query output
 - pipeline artifacts
-

Step 2 — Determine Business Impact

The data engineers must answer:

Which KPI is affected?

Settlement Rate
Settlement Gap
Merchant Exceptions

Is the issue:

- application failure?
 - data problem?
 - deployment problem?
 - query problem?
 - configuration problem?
 - pipeline problem?
-

12. Hidden SQL Injection

Injection A — Grain defect

A new query joins:

```
FACT_TRANSACTION  
  JOIN  
FACT_SETTLEMENT
```

before aggregation.

Result:

```
Settlement amount inflated
```

Injection B — Date defect

Version 2 accidentally filters using:

```
ingestion_ts
```

instead of:

```
transaction_ts
```

Result:

```
daily KPI values shift unexpectedly
```

Injection C — Configuration defect

Version 2 points to:

```
settlement_db_v2
```

instead of the approved production data source.

Injection D — API calculation defect

The API calculates:

```
settled_amount / transaction_count
```

instead of:

```
settled_amount / successful_transaction_amount
```

13. Incident Decision

The data engineers must decide:

Do we rollback or continue troubleshooting Version 2 in production?

They must provide evidence supporting the decision.

A strong incident response should contain:

```
Observed symptom
  ↓
Evidence collected
  ↓
Business impact
  ↓
Technical diagnosis
  ↓
Risk assessment
  ↓
Decision
  ↓
Rollback / Fix
  ↓
Validation
```

14. Rollback

If rollback is selected:

GREEN V2

↓
Traffic removed
↓
BLUE V1
↓
Smoke tests
↓
KPI validation
↓
Production restored

The data engineers must demonstrate:

Before rollback

Settlement Rate = 103.7%

After rollback

Settlement Rate $\approx$ expected baseline

The exact numbers should come from the trainer's dataset rather than being assumed.

15. Rollback Evidence

The data engineers must produce:

Incident timeline

10:00 - V2 deployment
10:08 - Blue-green cutover
10:15 - KPI anomaly detected
10:22 - Investigation started
10:35 - Root cause identified
10:40 - Rollback initiated
10:45 - V1 restored
10:50 - Smoke tests passed
10:55 - Business KPI reconciled

The timestamps are examples; data engineers must use their actual evidence.

Incident report

Incident ID:
Date:

Application:
Production Version:

What happened?

Business impact:

Detection method:

Technical root cause:

Evidence:

Decision:

Rollback performed:

Validation after rollback:

Preventive action:

Owner:

Due date:

16. Root Cause Analysis

The data engineers must perform a lightweight **5 Whys**.

Example structure:

Why was settlement rate >100%?
↓
Settlement amount was inflated.
↓
Why?
↓
Multiple settlement rows were joined to transactions.
↓
Why?
↓
Version 2 changed the aggregation logic.
↓
Why was this not detected?
↓
Regression test did not cover multiple settlements.
↓
Why?
↓
The test dataset contained only one settlement per transaction.

The data engineers must derive the actual chain from the injected incident.

17. Preventive Engineering Action

The data engineers must identify at least **3 permanent controls**.

Examples:

Control 1

Add a regression test for:

one transaction → multiple settlements

Control 2

Add a reconciliation gate:

Settlement amount
≤
Successful transaction amount

where that relationship is valid for the defined business rules.

Control 3

Add production KPI anomaly monitoring.

Control 4

Require SQL/data-model review for changes affecting Gold-layer metrics.

Control 5

Add automated blue-green smoke tests against business KPIs rather than only /health.

18. AFTERNOON — Leadership Presentation

Task 6 — Executive Presentation

Each data engineers gets **8–10 minutes**.

The presentation must answer:

Slide 1 — Business Problem

What problem does Bank of New York's Settlement Intelligence Platform solve?

Slide 2 — Architecture

Show:

```
Source
↓
Bronze
↓
Silver
↓
Gold
↓
API
↓
Dashboard
```

plus deployment architecture.

Slide 3 — Production Readiness

Show:

```
Tests ✓
Security ✓
Docker ✓
GitLab CI ✓
Jenkins ✓
Blue-Green ✓
```

Slide 4 — Production Incident

Explain:

- what happened
- how it was detected
- impact
- evidence

Slide 5 — Decision

Explain:

Why did you rollback or continue with the deployment?

Slide 6 — Recovery

Show:

```
V2
↓
Incident
↓
Detection
↓
Rollback
↓
V1
↓
Validation
```

Slide 7 — Prevention

Show the controls added to prevent recurrence.

Slide 8 — Business Outcome

Explain how the solution improves:

- settlement visibility
- operational investigation
- data trust
- deployment safety
- recovery capability

19. 360° Peer Review

Task 7 — Peer Review

Each data engineers reviews another data engineers/team.

Use a **1–5 scale**.

Area	1–2	3	4–5
Technical implementation	Major gaps	Functional	Strong
Data modelling	Significant issues	Adequate	Well justified
Testing	Limited	Reasonable	Comprehensive
Security	Major gaps	Basic controls	Strong controls
CI/CD	Incomplete	Working	Well engineered
Incident response	Unclear	Adequate	Evidence-driven
Communication	Difficult to follow	Clear	Structured
Business understanding	Weak	Good	Strong
Team collaboration	Limited	Good	Strong

Peer-review questions

Each reviewer must answer:

1. **What was technically strong?**
2. **What was the biggest engineering risk?**
3. **Was the production decision supported by evidence?**
4. **Was the rollback strategy adequate?**
5. **What one improvement would you recommend?**